INTERNAL DOCUMENTATION

ProjectPulse system guide

A project portfolio and agile analytics dashboard. It reads Jira-style records from PostgreSQL, computes PMO metrics in pandas, predicts which projects will be late, and writes plain-English diagnoses. This guide explains every page, every chart, and how the numbers get from the database to the screen.

8
dashboard pages

8
database tables

10,000
tasks in the sample data

0.80
ROC AUC of the delay model

Overview

What the system is and who it serves.

ProjectPulse is a Streamlit web app for a fictional 75-person company. It plays the role of a PMO reporting layer on top of Jira: projects, sprints, tasks, risks, scrum ceremonies and per-sprint performance.

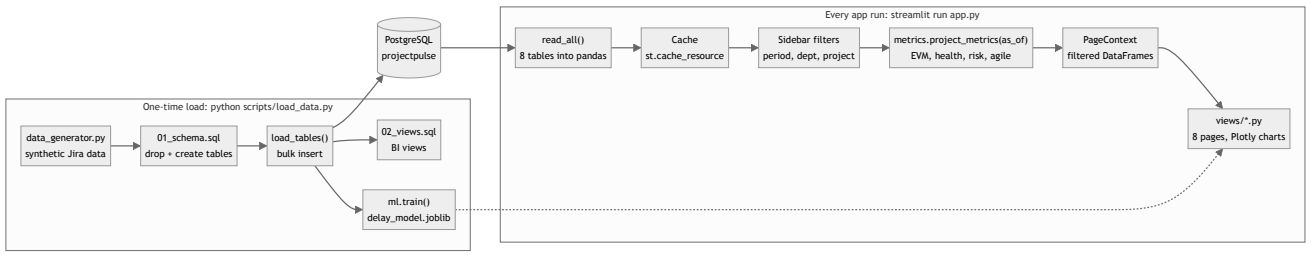
AUDIENCE	PAGES THEY USE MOST
Executives	Executive Dashboard, AI Insights
Project managers / PMO	Project Portfolio, Risk Management, Predictive Analytics
Resource managers	Resource Management
Scrum masters	Scrum Analytics
Team leads	Team Performance

Tech stack: Streamlit 1.66 (UI), PostgreSQL 18 on your Mac (SQLite fallback), SQLAlchemy 2 + psycopg 3 (database access), pandas and NumPy (all metrics), Plotly (charts), scikit-learn (delay model), joblib (model file).

> **Key design idea**
>
> Every metric is computed *as of* a date: the end of the date filter. Move the filter back to March 2026 and every page shows the portfolio exactly as it looked then. That is why the metrics are computed in Python rather than in fixed SQL views.

Architecture

Two separate flows: a one-time load, and the read path every time the app runs.



The app never writes to the database during normal use. The only write path from the UI is the **Generate & load demo data** button, which appears when the database is empty.

How data is pulled

What happens, in order, when you open the app or change a filter.

1. **Connection.** `projectpulse/config.py` reads `DATABASE_URL` from `.env`. If it isn't set, it uses the SQLite file `projectpulse.db`. `database.get_engine()` rewrites `postgresql://` to `postgresql+psycopg://` so SQLAlchemy uses psycopg 3.
2. **Readiness check.** `state.load_data()` calls `database_ready()`: all 8 tables must exist and `tasks` must have rows. On SQLite an empty database is filled automatically. On PostgreSQL you see an error with a button to generate the data.
3. **Full read.** `database.read_all()` runs `pd.read_sql_table()` for each of the 8 tables. There are no SQL filters or joins here; every row is loaded into memory. At about 12,500 rows in total this takes well under a second.
4. **Type cleanup.** `prepare()` converts date columns to `datetime64`, money and rates to floats, and nullable IDs (`tasks.sprint_id`, `tasks.employee_id`) to pandas `Int64`.
5. **Caching.** The table dictionary is cached with `@st.cache_resource`, keyed by a `data_version` counter in the session. Later page loads skip the database entirely.

6. **Filters.** `sidebar_filters()` turns the sidebar widgets into a `Filters` object with a start date, end date, departments and project IDs.
7. **Metrics.** `project_metrics(data, as_of=end)` builds one row per project with EVM, risk, agile and health numbers. It is cached with `@st.cache_data` per `as_of` date, so switching back to a date you've already viewed is instant.
8. **Page context.** `build_context()` slices every table down to the selected projects and dates and stores the result in `st.session_state.ctx`.
9. **Render.** The active page in `views/` calls `state.get_ctx()` and draws its KPI cards and Plotly charts from those DataFrames.

The Refresh button

The **Refresh** button at the bottom of the sidebar adds one to `data_version` and clears the metric caches. The next run re-reads every table from PostgreSQL and **retrains the delay model**, because `delay_model()` retrains whenever the version is above 0. Use it after you reload data with `load_data.py` while the app is running.

The SQL views

`sql/02_views.sql` creates four views: `vw_project_summary`, `vw_sprint_metrics`, `vw_employee_performance` and `vw_open_risks`. **The Streamlit app does not use them.** They exist for Tableau, Power BI, or ad-hoc SQL. Their numbers are close to the app's but not identical; see Gotchas.

Global filters

The sidebar filters apply to every page. Here is exactly what each one changes.

FILTER	OPTIONS	EFFECT ON THE DATA
Period	Last 3 / 6 / 12 months (default 12), Full history (24 months), Custom range	Sets **start** and **end**. For presets, end is the data's snapshot date (30 Sep 2026), not today's date. All "as of" metrics use **end**.
Department	Engineering, Data Analytics, Product Management, Marketing, Operations	Keeps projects owned by those departments and employees who belong to them. Empty means all.
Project	The 12 projects, narrowed by the department filter	Keeps only those projects. People are then limited to anyone who has tasks on them.
Reset		Back to the last 12 months, all departments and all projects.

How each table is sliced

TABLE	RULE IN `BUILD_CONTEXT()`
projects	Selected projects that had started by **end**
project metrics	Computed as of **end**, then limited to those projects
sprints	Sprints that overlap the window (start ≤ end and finish ≥ start)
scrum_ceremonies	Ceremonies dated between start and end
performance	Rows for the sprints above
tasks, risks	All rows for the selected projects; each page applies its own date logic, for example "open as of end"
employees	Selected departments; narrowed to people with tasks when specific projects are picked

Data model

Eight tables in PostgreSQL, defined in `sql/01_schema.sql` and mirrored by the ORM in `projectpulse/models.py`.

TABLE	ROWS	ONE ROW IS	KEY COLUMNS
employees	75	A person	department, role, hire_date, weekly_capacity_hours
projects	12	A project	project_manager_id, department, start / planned_end / actual_end dates, budget, actual_cost, status, priority
sprints	120	A sprint in a project	sprint_number, start / end dates, planned and completed story points, status
tasks	10,000	A Jira issue	task_key (e.g. PHX-142), issue_type, story_points, planned / actual hours, status, created / started / due / completion dates
risks	300	A risk register entry	category, probability 1–5, impact 1–5, risk_score = P × I, owner_id, status, mitigation_progress
scrum_ceremonies	500	A planning, review, retro or stand-up	ceremony_type, date, attendance_rate, duration_minutes, action_items

performance	1,089	One employee in one sprint	stories assigned / completed, story_points_completed, bugs_reopened, peer_review_score (out of 5)
project_history	480	A mid-project snapshot of an archived project (2019–2024)	The 5 model features plus `delayed` (0/1). Used only to train the model.

The schema enforces business rules with CHECK constraints: `risk_score = probability * impact`, valid status values, and Fibonacci story points. Foreign keys are indexed.

Pages and charts

Every page opens with six KPI cards, then its charts. Every table has an Export CSV button.

Executive Dashboard OVERVIEW

How healthy is the portfolio right now, and is it getting better or worse?

KPI cards

CARD	CALCULATION
Active Projects	Projects not completed as of end; subtitle shows the number in scope and completed
Portfolio Health	Budget-weighted average health score of active projects, with the change since the previous month-end
Budget Utilization	Actual cost to date ÷ budget, active projects
Delayed Projects	Finished late, or forecast to finish more than 10% of their planned duration late (at least 7 days)
Open Risks	Unresolved risks as of end, with the high-risk count
Resource Utilization	Assigned ÷ capacity hours in the window, for delivery staff who have any allocation

Charts

CHART	TYPE	WHAT IT SHOWS
Portfolio Health	Gauge	The portfolio health score from 0 to 100, in green, yellow and red bands.
Budget Trend	Bars + area	Planned vs actual spend. Switch between **To date** (cumulative) and **Monthly**. Planned spreads each budget evenly over the project's planned days. Actual assigns each project's cost to completed tasks in proportion to their hours.
Project Status Distribution	Bar	Count of projects by computed status: Completed, Delayed, At Risk or On Track. These are derived from the numbers, not the PM's reported status.
Risk Breakdown	Stacked bar	Open risks by category, split into Low, Medium and High severity.
Department Comparison	3 small bar charts	Average health score, CPI and open risks for each department.

Portfolio Health Trend	Line	Budget-weighted health at each month-end in the window. Every point re-runs the full metric engine as of that date.
Needs Attention	Table	The 8 active projects with the lowest health, with RAG, SPI and CPI. Export gives the full portfolio summary.

Project Portfolio OVERVIEW

Which projects are in trouble, and why?

KPI cards: number of projects; Green, Yellow and Red counts; total cost variance (EV − AC); total schedule variance (EV − PV).

CHART	TYPE	WHAT IT SHOWS
Portfolio Register	Table	Every project with health, RAG, progress, SPI, CPI, budget, cost and risks. It shows both **Status** (computed) and **Reported** (PM's status), and notes any projects where they disagree.
Health Score Composition	Stacked bar	Each project's score split into its weighted parts: 40% schedule, 30% budget and 30% risk, adding up to 100.
Schedule vs Cost Performance	Bubble scatter	SPI on x, CPI on y, bubble size by budget. Dotted lines at 1.0 split the chart into four quadrants; bottom-left means behind schedule and over budget.
Project Drill-down	Metrics + 2 charts	Pick a project to see PV, EV, AC, EAC, VAC and forecast finish. Includes a **Timeline** (start, planned end, forecast end) and **Health components** bars from 0 to 100.

Resource Management DELIVERY

Who is overloaded, who has spare capacity?

KPI cards: average utilization; overloaded (above 100%); under-utilised (below 70%); on the bench (no allocation); total capacity hours in the period; average projects per person. Project managers are excluded from utilization.

CHART	TYPE	WHAT IT SHOWS
Workload Heatmap	Heatmap	Rows are people, columns are months, and colour is utilization %. A department dropdown narrows the rows.
Department Utilization	Bar	Assigned ÷ capacity hours per department over the period.
Capacity vs Assignment	Bar + tick	The 20 most-allocated people. The bar is assigned hours and the tick is capacity, so a bar past its tick means overloaded.
Resource Register	Table	Every delivery team member sorted by utilization.

> **How utilization is measured**
>
> Each person's planned task hours in a sprint are spread evenly over that sprint's working days (`daily_assignment()`). Capacity = weekly capacity ÷ 5 × working days in the period.

Risk Management DELIVERY

Where is the risk exposure, and are risks being closed?

KPI cards: open risks (with raised and resolved in the period); high-risk items (score ≥ 15); average mitigation progress (only when the window ends on the latest date); closure rate = resolved ÷ (resolved + open); risk exposure = Σ score of open risks; average age of open risks.

CHART	TYPE	WHAT IT SHOWS
Risk Matrix	5×5 heatmap	Count of open risks in each probability × impact cell. The red outline marks the high-severity zone.
Probability vs Impact	Scatter	One dot per open risk, slightly jittered so dots don't stack. Size and colour show severity.
Risk Trends	Stacked bars + lines	Open risks at each month-end by severity, with lines for risks raised and resolved that month.
Exposure by Project	Bar	Sum of open risk scores per project.
Top Risks	Table	Unresolved risks ranked by score, then age, with owner and mitigation %.

Severity bands: High ≥ 15, Medium 8–14, Low < 8.

Scrum Analytics DELIVERY

Are teams predictable and are ceremonies working?

KPI cards: average sprint velocity; commitment reliability; ceremony attendance; average cycle time; average lead time; sprint success rate (share of sprints with reliability ≥ 85%).

CHART	TYPE	WHAT IT SHOWS
Velocity Trend	Grouped bars	Committed vs completed story points per sprint for one project (defaults to Project Phoenix).
Sprint Burndown	Line + area	Remaining story points each day of a chosen sprint against the ideal straight line.
Sprint Success Rate by Project	Bar	Share of each project's closed sprints that met the 85% commitment threshold.
Story Point Completion Trend	Area	Story points completed per month across all projects in scope.
Ceremony Attendance Trend	Lines	Average monthly attendance for planning, review, retro and stand-up.
Ceremony Summary	Table	Counts, attendance, duration and action items by ceremony type. Stand-ups are sampled, not every day.

Team Performance DELIVERY

Who delivers reliably, and how do departments compare?

KPI cards: task completion rate; story points delivered; on-time delivery; defect rate (bugs reopened ÷ stories completed); average peer review score (out of 5); average performance score.

CHART	TYPE	WHAT IT SHOWS
Top Performers	Stacked bar	The top 10 people by score, each bar split into its four weighted parts.
Performance Distribution	Histogram	Number of people in each 4-point score band.
Department Comparison	Bar	Average performance score per department.
Department KPIs	Table	Averages of each KPI per department.
Employee Ranking	Table	Everyone in scope ranked by score. Bands: Exceeds ≥ 85, Meets ≥ 72, otherwise Developing.

Predictive Analytics INTELLIGENCE

Which active projects are likely to finish late?

KPI cards: ROC AUC, accuracy, precision, recall and F1, all on the held-out test set; and the number of active projects in the High Risk class.

CHART	TYPE	WHAT IT SHOWS
Active Project Predictions	Bar + table	Delay probability for each active project, coloured Low (< 35%), Medium (35–65%) or High (≥ 65%). The table lists the five input features.
Confusion Matrix	Heatmap	Predicted vs actual on the 120 test projects, at a 0.50 threshold.
ROC Curve	Area line	True positive vs false positive rate across thresholds.
Feature Importance	Bar	How much ROC AUC drops when each feature is shuffled (permutation importance).
Prediction Interface	Sliders, gauge, bar	Start from a project or the typical project, move the five sliders, and see the new probability. **What moves the prediction** shows how far each feature pushes the result compared with a typical value.

AI Insights INTELLIGENCE

"Why is Project Phoenix unhealthy?"

KPI cards: needs intervention (Red); on watch (Yellow); high-severity risks; overloaded people; forecast overrun (Σ EAC − budget); the project with the highest delay risk.

PANEL	WHAT IT DOES
Executive Briefing	A generated portfolio summary: Red and Yellow projects, top risks and budget position, plus "Priority actions this week" (the top recommendation for each non-Green project). Download it as Markdown.
Project Diagnosis	Pick a project to see its health headline, ranked drivers, what is working, and a recommendation for each driver.
Ask ProjectPulse	A chat box. Questions are routed by keywords to an answer generated from the metrics. See AI Insights engine.

Metric formulas

All defined in `projectpulse/metrics.py`. Weights and thresholds live in `projectpulse/config.py`.

Earned value (EVM)

METRIC	FORMULA	READING IT
% complete	done story points ÷ total story points	Actual progress
Planned %	linear from first sprint start to planned end	Where the plan says you should be. The initiation period before sprint 1 counts as 0%.
PV	budget × planned %	Planned value
EV	budget × % complete	Earned value
AC	cost rate × actual hours of done tasks	Actual cost to date. Cost rate = project actual_cost ÷ total done hours.
SPI	EV ÷ PV	Below 1 means behind schedule. Completed projects use planned ÷ actual duration.

CPI	EV ÷ AC	Below 1 means over budget
SV / CV	EV − PV / EV − AC	Negative is bad
EAC / VAC	budget ÷ CPI / budget − EAC	Forecast final cost and the forecast over- or underspend
Forecast finish	as_of + remaining ÷ delivery rate so far	Capped at 3 years. Early in a project it assumes the plan.

Health score

```
health = 0.40 × schedule_health + 0.30 × budget_health + 0.30 × risk_health

schedule_health = 100 - 250 × max(0, 1 - SPI)          # -2.5 pts per 1% behind
budget_health   = 100 - 300 × max(0, 1/CPI - 1)        # -3 pts per 1% overrun
risk_health     = 100 × e^(-exposure / 200)            # 50 → 78, 100 → 61, 200 → 37
exposure        = Σ score of unresolved risks; "Mitigating" risks × (1 - mitigation% ÷ 200)
```

Green ≥ 80 | Yellow 60–79 | Red < 60

Computed status is checked in this order: Completed if finished, Delayed if `is_delayed`, At Risk if not Green, otherwise On Track.

Agile, resources and people

METRIC	FORMULA
Velocity	completed story points in a sprint
Velocity change	mean of last 3 closed sprints vs the 3 before them
Commitment reliability	completed ÷ planned SP (last 3 closed sprints); success ≥ 85%
Lead / cycle time	completion − created / completion − started (days)
Utilization	planned sprint hours ÷ (weekly capacity ÷ 5 × working days)
Team utilization	same, over the last 8 weeks, for everyone on the project
Attendance	mean ceremony attendance, last 90 days
Performance score	30% completion + 30% on-time + 20% contribution + 20% peer review ÷ 5
Sprint contribution	person's SP ÷ team-average SP in the same sprint, capped at 1.5×

Defect rate	bugs reopened ÷ stories completed

Thresholds you can tune in `config.py`

SETTING	VALUE	USED FOR
HEALTH_WEIGHTS	0.40 / 0.30 / 0.30	Schedule / budget / risk weights
HEALTH_GREEN / YELLOW	80 / 60	RAG bands
PERFORMANCE_WEIGHTS	0.30 / 0.30 / 0.20 / 0.20	Employee score
OVERLOAD_THRESHOLD	1.00	Overloaded person
UNDERUTILISED_THRESHOLD	0.70	Under-utilised person
SPRINT_SUCCESS_THRESHOLD	0.85	Successful sprint
HIGH_RISK_SCORE	15	High-severity risk
DELAY_CLASS_BANDS	0.35 / 0.65	Low / Medium / High delay risk

Delay prediction

`projectpulse/ml.py` : a classifier that estimates the chance a project finishes after its planned end date.

1. **Training data.** 480 snapshots in `project_history`, labelled `delayed` = 1 or 0.
2. **Features (5).** Budget utilization (AC ÷ EV), open risks, average resource utilization, sprint velocity, tasks remaining.
3. **Model selection.** Logistic Regression, Random Forest and Gradient Boosting are compared with stratified 5-fold cross-validated ROC AUC on 75% of the data.
4. **Evaluation.** The winner is retrained and tested once on the remaining 25% (120 projects). Your last run picked Logistic Regression: ROC AUC 0.798, accuracy 0.692.
5. **Saved.** The model, metrics, ROC data, importance and training medians are saved to `models/delay_model.joblib`.
6. **Scoring.** `current_features()` maps each active project's live metrics onto the five features. A missing value, such as no closed sprint yet, is filled with the training median.
7. **Explaining.** `local_drivers()` resets one feature at a time to its typical value and measures how much the probability changes.

> **Caveat**
> The training archive is synthetic, generated from a known formula plus noise. The model demonstrates the workflow; its accuracy says nothing about real projects.

AI Insights engine

`projectpulse/insights.py` : rule-based, with no LLM and no API key.

`diagnose()` checks each project against the rules below. Each rule that fires becomes a **driver** with a severity from 0 to 1 and a recommendation. Drivers are sorted by severity and the top 6 are shown.

DRIVER	FIRES WHEN
Schedule	SPI < 0.95
Velocity decline	velocity change ≤ −8%
Commitment	reliability < 85%
Budget overrun	budget variance > 3%
High risks	any open risk with score ≥ 15
Team overload	team utilization > 100%
Attendance	ceremony attendance < 85%
Blocked work	10 or more blocked tasks
Predicted delay	model delay probability ≥ 35% (active projects)

How the chat routes questions

The question is matched against keyword patterns in this order, and the first match wins. Project names are found by their short name, for example "phoenix".

INTENT	TRIGGER WORDS	EXAMPLE
compare	compare, vs, versus	Compare Orion and Aurora
budget	budget, cost, spend, CPI, overrun	Which projects are over budget?
risk	risk	What are the top risks on Titan?

resource	team, resource, capacity, overload, workload	Who is overloaded?
agile	velocity, sprint, scrum, burndown	How is Phoenix's velocity?
delay	delay, late, predict, forecast, deadline	Which projects will be late?
portfolio	portfolio, overall, summary, worst, best	Give me a portfolio summary
health	why, health, status, problem	Why is Project Phoenix unhealthy?

Synthetic data

`projectpulse/data_generator.py`: why the numbers tell a coherent story.

Each of the 12 projects has a hidden **health profile**. The profile drives every downstream behaviour, so the dashboards show causes and effects that line up.

PROFILE	RELIABILITY	VELOCITY TREND	TEAM LOAD	RISK WEIGHT	ATTENDANCE	COST C
healthy	93%	0%	94%	1.0×	93%	
watch	88%	−8%	108%	1.5×	87%	
troubled	87%	−24%	135%	2.4×	80%	

Sprint story points are summed from their tasks, cost follows earned progress, and performance rows are aggregated from tasks, so totals match across pages. The generator is seeded by `PP_SEED` (default 42), so the same seed always gives the same data. `PP_AS_OF` (default 2026-09-30) sets the snapshot date and the 24 months of history before it.

Projects: Mercury, Vanguard, Atlas, Helios, Titan, Phoenix, Quantum, Orion, Nebula, Aurora, Horizon and Sentinel.

Code map

Where to look when you want to change something.

FILE	RESPONSIBILITY	CHANGE IT TO...
app.py	Entry point: page config, navigation, sidebar, Refresh button	Add or reorder pages
projectpulse/config.py	Env settings and business rules	Tune weights and thresholds

projectpulse/database.py	Engine, schema build, bulk load, reads	Change how data is read
projectpulse/models.py	SQLAlchemy ORM, the same tables as the SQL DDL	Add a column (update both)
projectpulse/metrics.py	All formulas	Change a metric definition
projectpulse/ml.py	Train, evaluate, score, explain	Add features or models
projectpulse/insights.py	Diagnosis rules and chat router	Add a driver or intent
projectpulse/data_generator.py	Synthetic data	Change volumes or project mix
projectpulse/ui/state.py	Loading, caching, filters, PageContext	Add a global filter
projectpulse/ui/components.py	KPI cards, chart card, CSV export, formatting	Change shared UI pieces
projectpulse/ui/theme.py	Colours, Plotly template, CSS	Restyle charts
views/*.py	One file per page	Add or change a chart
sql/01_schema.sql	PostgreSQL DDL	Schema changes
sql/02_views.sql	BI views (not used by the app)	Tableau / Power BI feeds
sql/03_sample_queries.sql	Example PMO questions in SQL	Learning and ad-hoc analysis
scripts/	generate_data, load_data, train_model	Command-line jobs
tests/test_projectpulse.py	36 tests	Run with `pytest -q`

Running and operating

Your setup: PostgreSQL 18 (EDB installer) on port 5432, database and user both named `projectpulse`.

Day-to-day

```
cd "/Users/kabitabandana/Documents/Tribhuwan/Projects US/PMO System"
source .venv/bin/activate
streamlit run app.py                        # http://localhost:8501
```

Commands

COMMAND	WHAT IT DOES
`python scripts/load_data.py`	**Drops and recreates** all tables, loads fresh data, creates the views and retrains the model (about 6 s)
`python scripts/load_data.py --seed 7`	A different but reproducible data set
`python scripts/load_data.py --skip-model`	Load data without retraining
`python scripts/train_model.py`	Retrain from the database and print the model report
`python scripts/generate_data.py --out data/`	Export every table to CSV without touching the database
`pytest -q`	Run the test suite
`psql -U projectpulse -h localhost -d projectpulse`	Open a SQL prompt on the app database

Environment variables (`.env`)

VARIABLE	YOUR VALUE	MEANING
DATABASE_URL	postgresql://projectpulse:... @localhost:5432/projectpulse	Unset means SQLite file `projectpulse.db`
PP_SEED	42	Random seed for the generator and model
PP_AS_OF	2026-09-30	Snapshot date for the generated history

Troubleshooting

SYMPTOM	CAUSE AND FIX
role "..." does not exist	The user in `DATABASE_URL` wasn't created. Create it as `postgres` with `CREATE ROLE … LOGIN PASSWORD …`
password authentication failed	Wrong password in `.env`, or you're using your Mac username instead of `postgres` / `projectpulse`

Connection refused	The server is down. Run `cd /tmp && sudo -u postgres /Library/PostgreSQL/18/bin/pg_ctl -D /Library/PostgreSQL/18/data start`
"The database is empty"	Run `python scripts/load_data.py` or press the button on screen
Numbers don't change after a reload	Press **Refresh** in the sidebar, or restart Streamlit
Model error after upgrading scikit-learn	Delete `models/delay_model.joblib`; it retrains on the next start

Gotchas

Things that are easy to trip over.

load_data.py is destructive

It drops every table before loading. Any data you added by hand is lost. The Docker image runs it on every container start.

Password checks on your PostgreSQL

During setup, `pg_hba.conf` was set to `trust`. If you haven't switched it back to `scram-sha-256` and restarted, any local user can open any database without a password.

SQL views differ from the dashboard

`vw_project_summary` uses a simpler planned % (no initiation period), counts risk exposure without the mitigation discount, and uses each risk's current status rather than "as of" a date. Expect small differences from the app's SPI and health.

"Today" is the data's date

Period presets end on the latest activity in the data (30 Sep 2026), not on the real calendar date.

Everything loads into memory

The app reads every row of every table. That is fine at about 12,500 rows. Connecting real Jira data with millions of rows would need filtered SQL reads.

Git repository location

Your Git repository currently starts at your home folder, not at this project. Run `git init` inside the project folder before pushing to GitHub.
